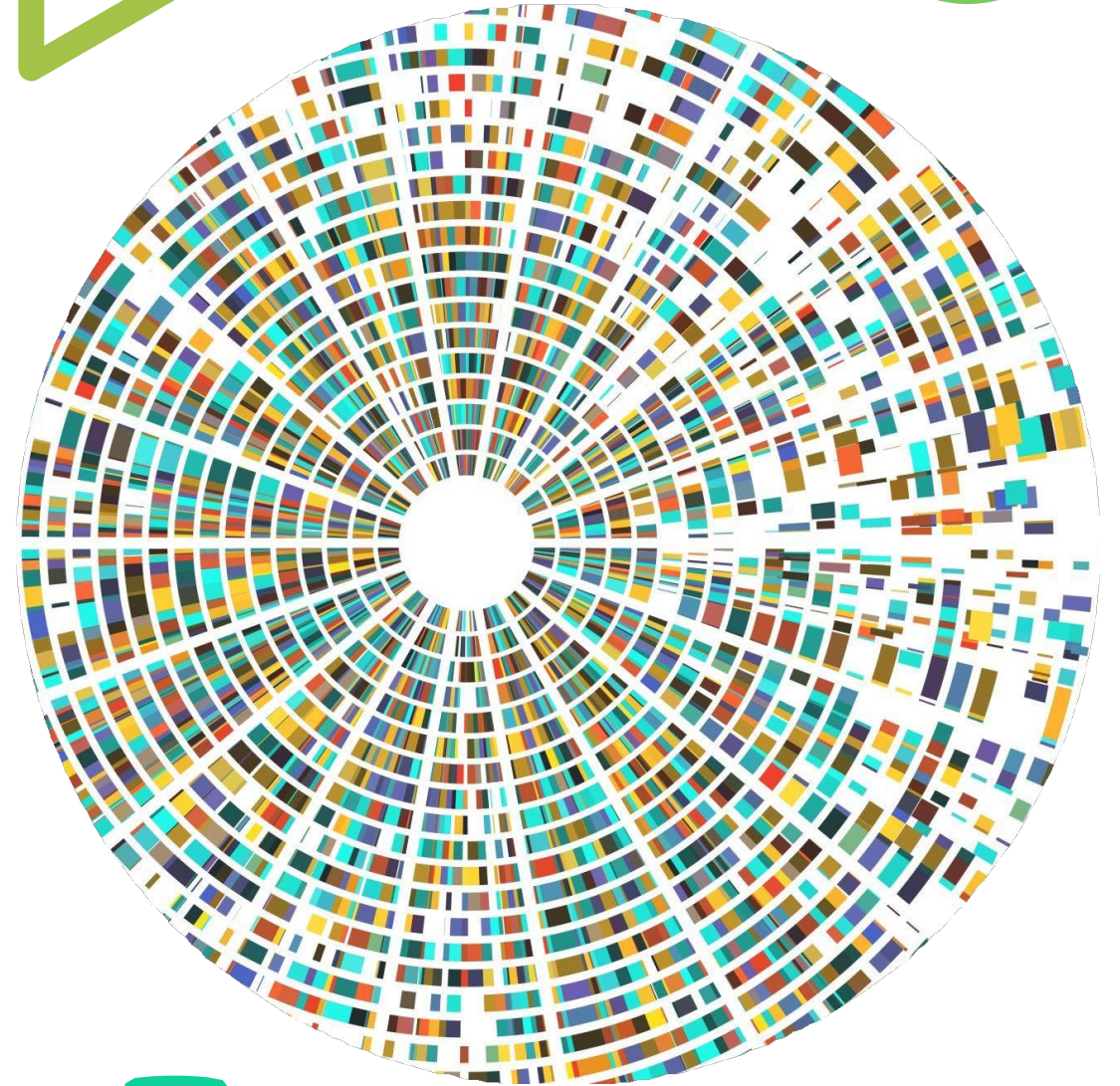


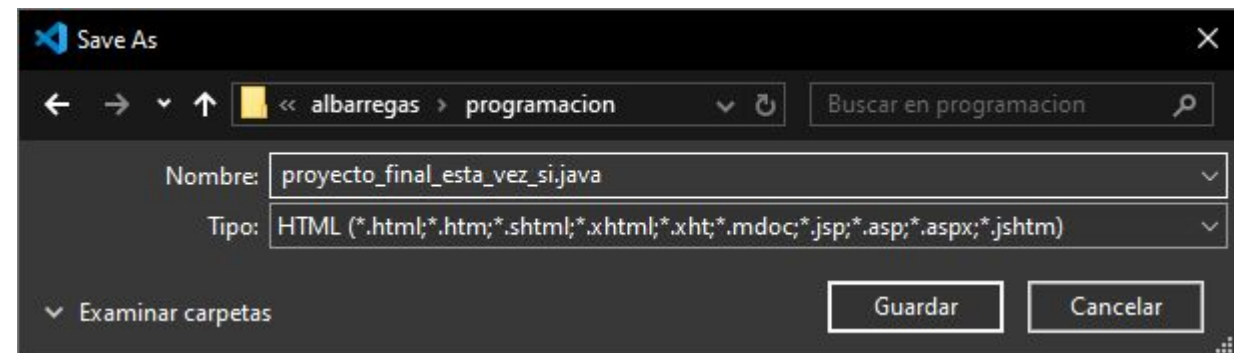
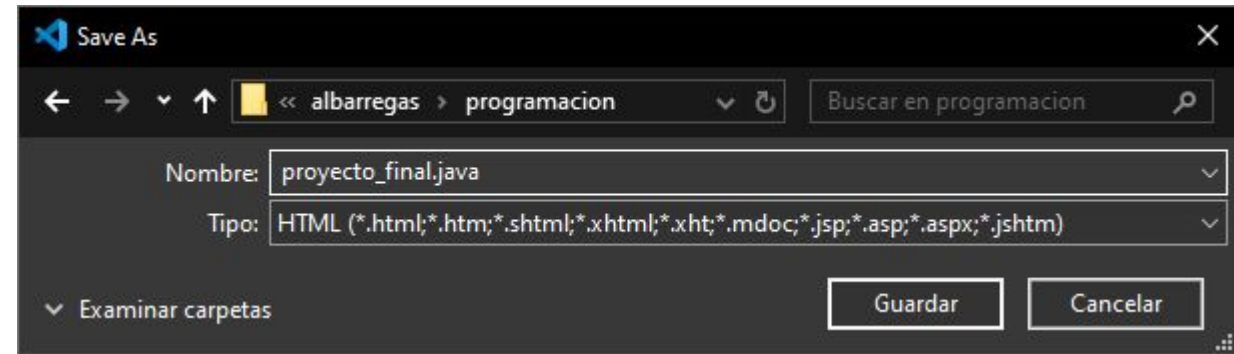
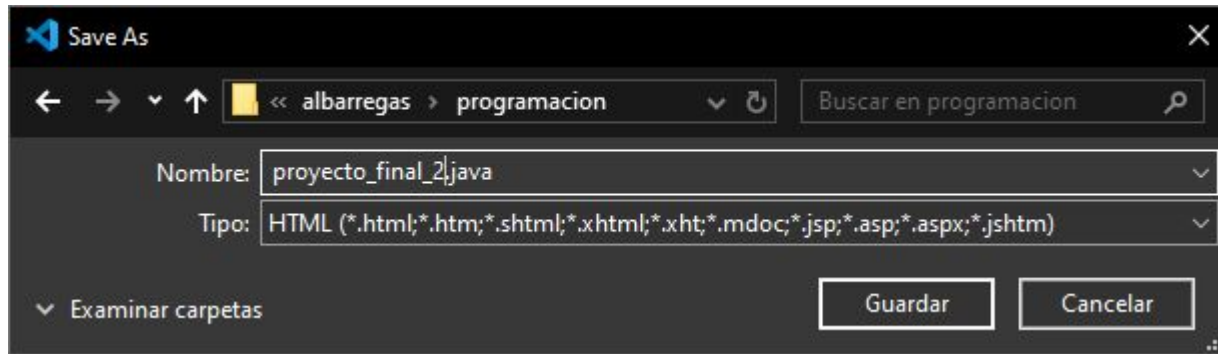
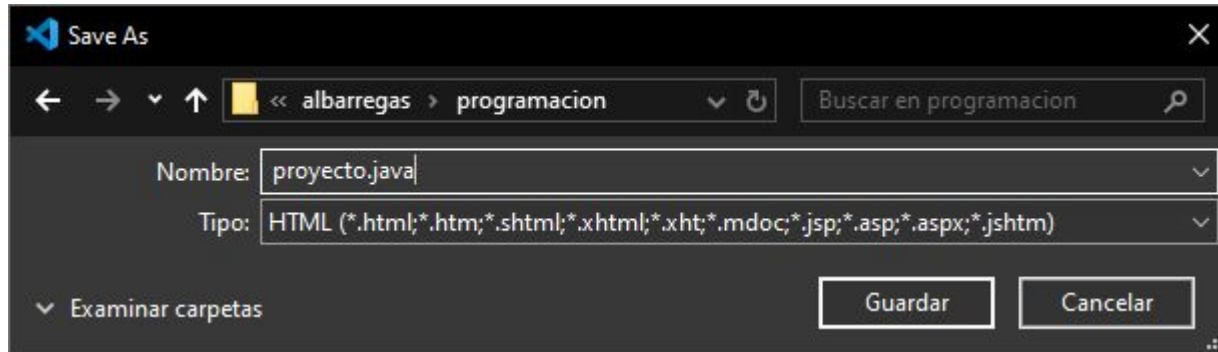


Gestión de repositorios

1º DAM/DAW
(2024-2025)



Guardando versiones de un proyecto



Sistemas de control de versiones (VSC)



Mantiene el historial de los cambios realizados en un repositorio.

VSC vs Repositorio



Comandos y algoritmos para el control de versiones.



Espacio de almacenamiento en la nube.
Repositorio remoto

Descargar git

 **git** --distributed-is-the-new-centralized

Type / to search entire site...

About
Documentation
Downloads
GUI Clients
Logos
Community

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to [read online for free](#). Dead tree versions are available on [Amazon.com](#).

Downloads

 **macOS** **Windows**
 **Linux/Unix**

Latest source Release
2.47.0
[Release Notes](#) (2024-10-06)
[Download for Windows](#)

Older releases are available and the [Git source repository](#) is on GitHub.

GUI Clients

Git comes with built-in GUI tools (**git-gui**, **gitk**), but there are several third-party tools for users looking for a platform-specific experience.

[View GUI Clients →](#)

Logos

Various Git logos in PNG (bitmap) and EPS (vector) formats are available for use in online and print projects.

[View Logos →](#)

Git via Git

If you already have Git installed, you can get the latest development version via Git itself:

```
git clone https://github.com/git/git
```

You can also always browse the current contents of the git repository using the [web interface](#).

Herramientas - consola de git

```
MINGW64:/c/Users/me/git
me@work MINGW64 ~
$ git clone https://github.com/git-for-windows/git
Cloning into 'git'...
remote: Enumerating objects: 500937, done.
remote: Counting objects: 100% (3486/3486), done.
remote: Compressing objects: 100% (1415/1415), done.
remote: Total 500937 (delta 2494), reused 2917 (delta 2071), pack-reused 497451
Receiving objects: 100% (500937/500937), 221.14 MiB | 1.86 MiB/s, done.
Resolving deltas: 100% (362274/362274), done.
Updating files: 100% (4031/4031), done.

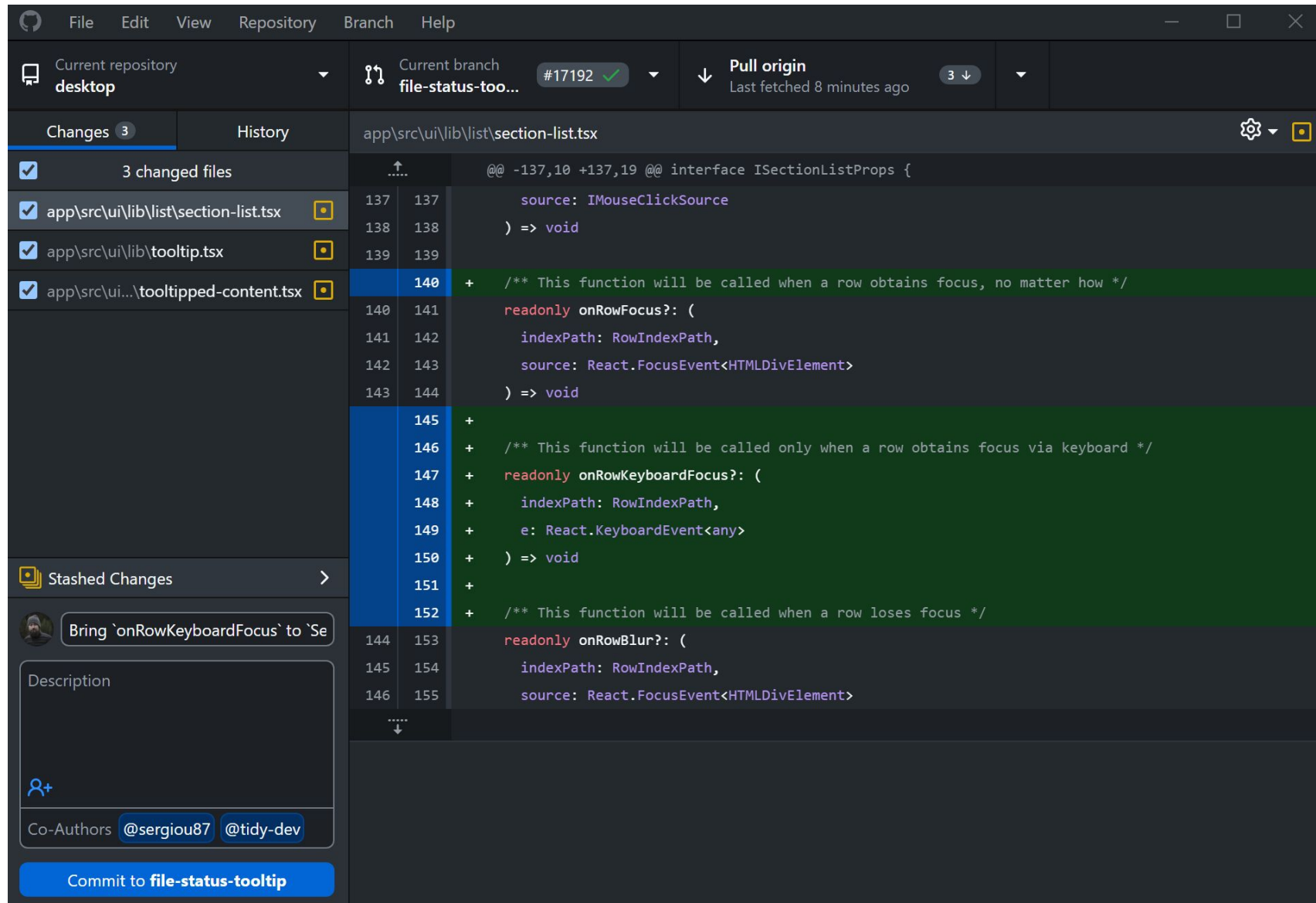
me@work MINGW64 ~
$ cd git

me@work MINGW64 ~/git (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

me@work MINGW64 ~/git (main)
$ |
```

Herramientas - github desktop



Herramientas - gitkraken (de pago)

The screenshot displays the GitKraken application interface, which is a paid tool for managing Git repositories. The interface is divided into several panels:

- Left Panel:** Contains a sidebar with navigation options like 'workspace', 'repository', and 'branch'. It also shows a list of pull requests and issues, along with a 'JIRA ISSUES' section.
- Central Panel:** Displays a graphical view of the repository's history, showing branches (main, 17-x-y, etc.) and commits. The '17-x-y' branch is currently selected.
- Right Panel:** Shows the '4 file changes on 17-x-y' section, including a list of files (main.js, ELECTRON_VERSION) and a 'Commit Message' field for staging changes.

The interface is dark-themed and includes various icons and buttons for navigating between different views and performing Git operations. The status bar at the bottom indicates the current version (8.2.0-RC14) and provides links to feedback and the PRO version.

Herramientas - sourcetree

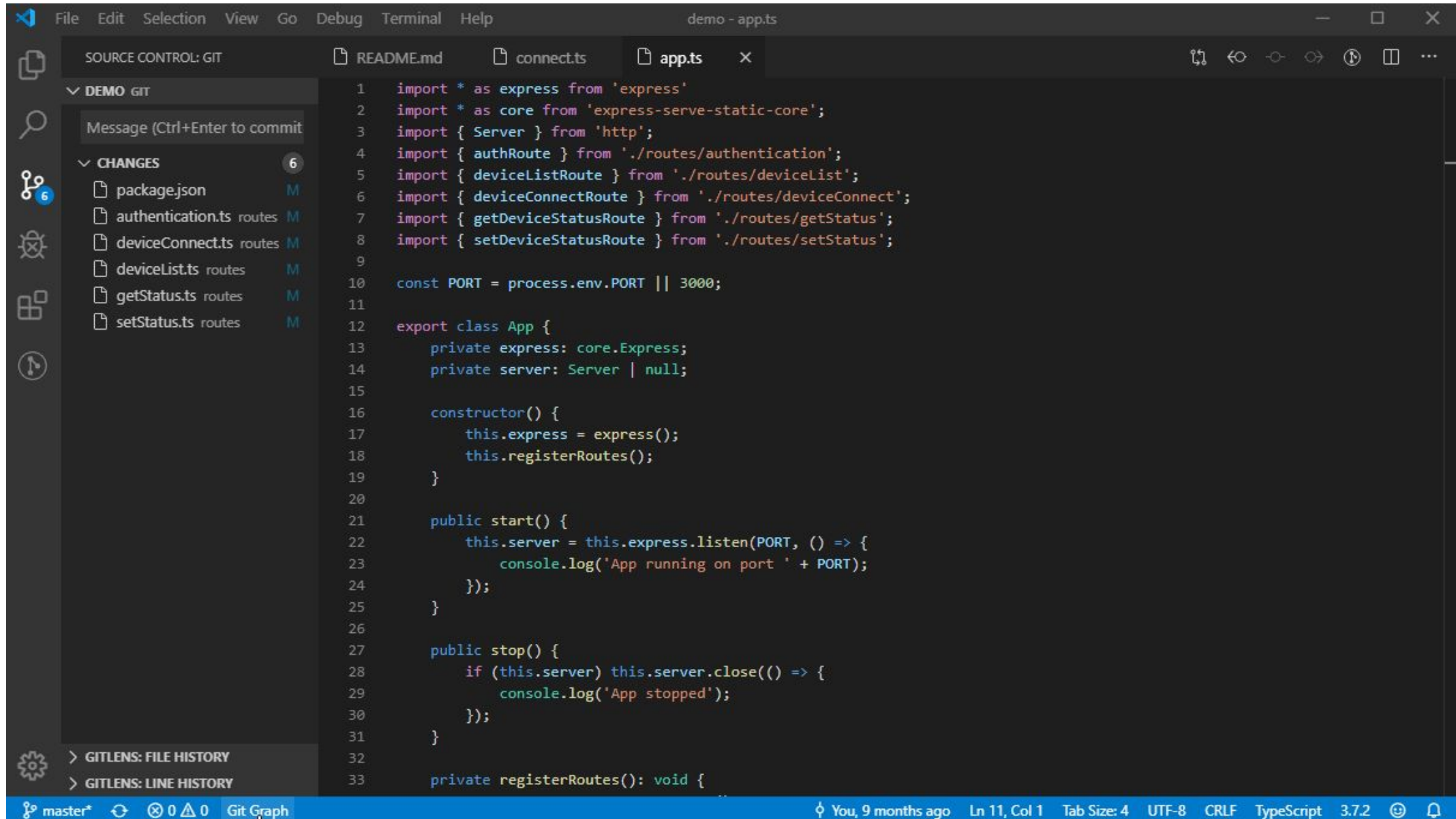
The screenshot displays the Sourcetree application interface for a repository named 'jquery-ui-carousel'. The interface is divided into several sections:

- Top Bar:** Contains menu items (File, Edit, View, Repository, Actions, Tools, Help) and a toolbar with icons for Commit, Push, Pull, Fetch, Branch, Merge, Stash, Discard, and Tag. It also includes links to Git Flow, Terminal, Explorer, and Settings.
- Left Panel:** Shows the 'FILE STATUS' section with 'Working Copy', 'BRANCHES' (listing 'master'), and 'TAGS' (listing various versions like 0.10.7, 0.11.0, etc.).
- Central Panel:** Displays a commit history table with columns for Graph, Description, Date, Author, and Commit. The current commit is 'Add imagesloaded js from http:' by Anton Evers, dated 17 Apr 2013 7:04.
- Bottom Panel:** Shows the 'Commit' details for the selected commit, including the commit hash, author, date, and a list of files changed (demo.html, js/lib/jquery.imagesloaded.js).
- Right Panel:** Displays the 'File Contents' for the selected file 'js/lib/jquery.imagesloaded.js', showing the JavaScript code.

Graph	Description	Date	Author	Commit
o master origin/master origin/HEAD	Add imagesloaded js from http:	17 Apr 2013 7:04	Anton Evers <anto	a06026a
	Add images in the demo so I can see what this carousel does with image width during im	17 Apr 2013 7:00	Anton Evers <anto	b42db55
	pagination and next / prev action class updates	12 Mar 2013 17:44	Richard Scarrott <i	48c2c13
	Merge pull request #55 from gepoch/disable-buttons	12 Mar 2013 17:03	Richard Scarrott <i	4ea27b0
	Disable both the left and right buttons when the widget is disabled.	8 Mar 2013 14:26	George Marchin <i	422746d
	0.11.1 Corrected event namespacing	26 Feb 2013 16:20	Richard Scarrott <i	b88d8d5
	origin/gh-pages Create gh-pages branch via GitHub	25 Feb 2013 18:28	Richard Scarrott <i	84b3ecb
	0.11.0 Updated jquery manifest for latest touch release	25 Feb 2013 18:19	Richard Scarrott <i	2db3b5c
	Touch support	25 Feb 2013 17:32	Richard Scarrott <i	57f4fdb
	Added translate3d option and prevented callbacks from firing during setup	25 Feb 2013 14:38	Richard Scarrott <i	7dab8ec
	Create gh-pages branch via GitHub	26 Jan 2013 18:53	Richard Scarrott <i	00906c7
	Updated dependencies in jQuery manifest	26 Jan 2013 18:21	Richard Scarrott <i	9257364

```
+ /!*
+  * jQuery imagesLoaded plugin v2.1.1
+  * http://github.com/desandro/imagesloaded
+  *
+  * MIT License. by Paul Irish et al.
+  */
+
+ /*shhint curly: true, eqeqeq: true, noempty: true,
+ /*global jQuery: false */
+
+ ;(function($, undefined) {
+ 'use strict';
+ }
```

Herramientas - gitgraph



The image shows a screenshot of the Visual Studio Code editor interface. The main editor window displays a TypeScript file named `app.ts` with the following code:

```
1 import * as express from 'express';
2 import * as core from 'express-serve-static-core';
3 import { Server } from 'http';
4 import { authRoute } from './routes/authentication';
5 import { deviceListRoute } from './routes/deviceList';
6 import { deviceConnectRoute } from './routes/deviceConnect';
7 import { getDeviceStatusRoute } from './routes/getStatus';
8 import { setDeviceStatusRoute } from './routes/setStatus';
9
10 const PORT = process.env.PORT || 3000;
11
12 export class App {
13   private express: core.Express;
14   private server: Server | null;
15
16   constructor() {
17     this.express = express();
18     this.registerRoutes();
19   }
20
21   public start() {
22     this.server = this.express.listen(PORT, () => {
23       console.log('App running on port ' + PORT);
24     });
25   }
26
27   public stop() {
28     if (this.server) this.server.close(() => {
29       console.log('App stopped');
30     });
31   }
32
33   private registerRoutes(): void {
```

The left sidebar shows the 'SOURCE CONTROL: GIT' view. Under 'DEMO GIT', there is a 'Message (Ctrl+Enter to commit)' input field. Below that, the 'CHANGES' section shows a list of files that have been modified (M):

- package.json
- authentication.ts routes
- deviceConnect.ts routes
- deviceList.ts routes
- getStatus.ts routes
- setStatus.ts routes

The bottom status bar shows the 'Git Graph' view, indicating the current commit is 'master*' and the author is 'You, 9 months ago'. The status bar also displays 'Ln 11, Col 1', 'Tab Size: 4', 'UTF-8', 'CRLF', 'TypeScript', and '3.7.2'.

Configuración inicial de GIT



```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```



```
$ git config --list
user.name=John Doe
user.email=johndoe@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
```

Configuración inicial de GIT - GitGraph

Branches: Show All ☒ Show Remote Branches

Graph

	Description	Date	Author	Commit
Uncommitted Changes (3)				
○	main Merge branch 'feature/funcion3'	16 Oct 2024 1...	Serra	345849c4
●	feature/funcion3 New feature 3	16 Oct 2024 1...	Serra	9a487abf
●	feature/funcion2 New feature 2	16 Oct 2024 1...	Serra	35b91021
●	feature/funcion1 New feature	16 Oct 2024 1...	Serra	4fa408b8
	First commit	16 Oct 2024 1...	Serra	0d372ff2
	Add gitignore	16 Oct 2024 1...	Serra	bf17c208

Repository Settings

General

Name: ejemplo

Initial Branches: Show All

☒ Show Stashes

☒ Show Tags

☐ Include commits only mentioned by reflogs ⓘ

☐ Only follow the first parent of commits ⓘ

User Details

User Name:

User Email:

Edit Remove

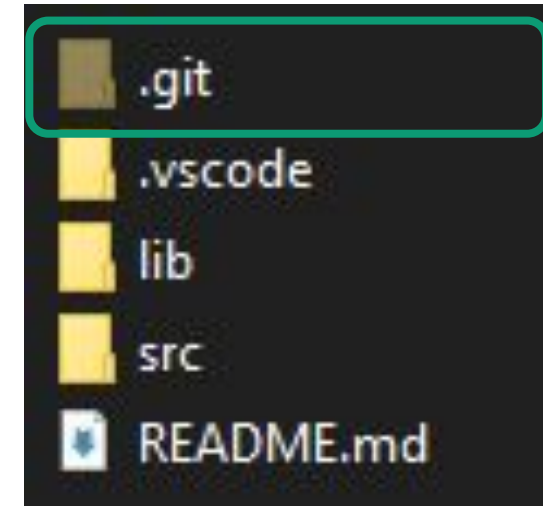
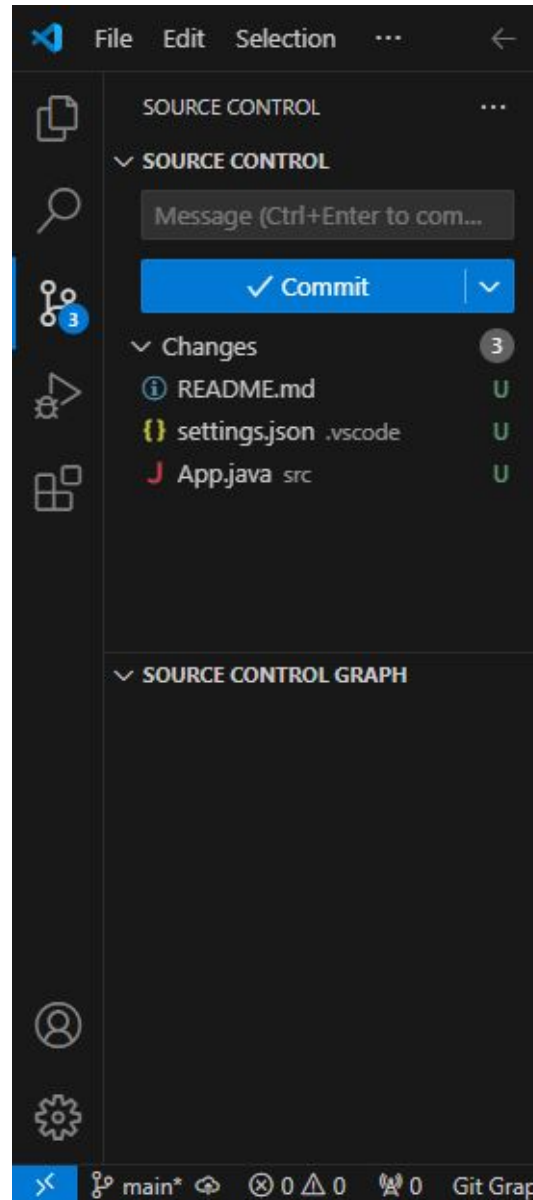
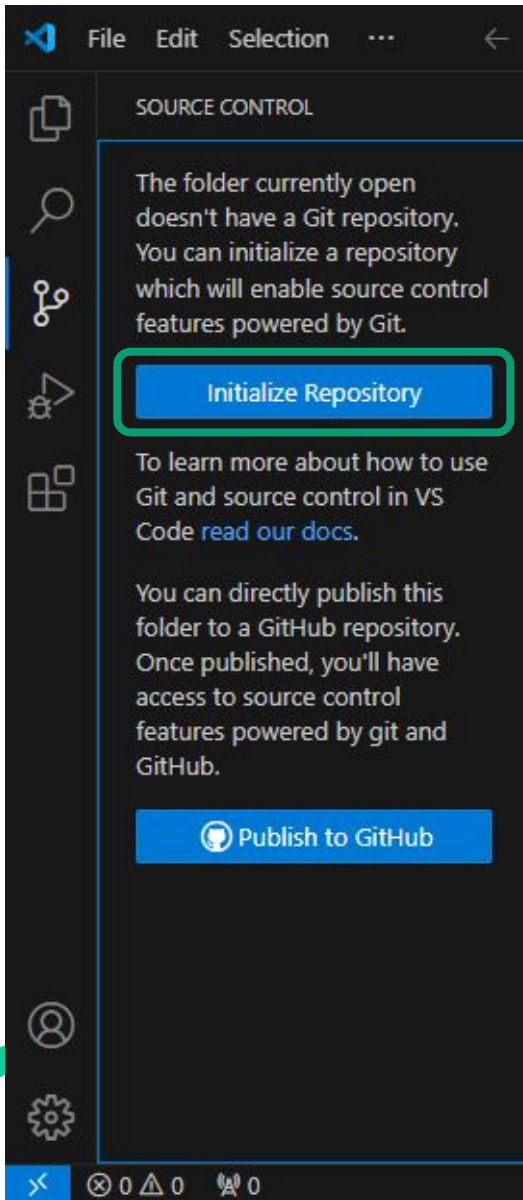
Remote Configuration

Remote	URL	Type	Action
There are no remotes configured for this repository.			
+ Add Remote			

Issue Linking

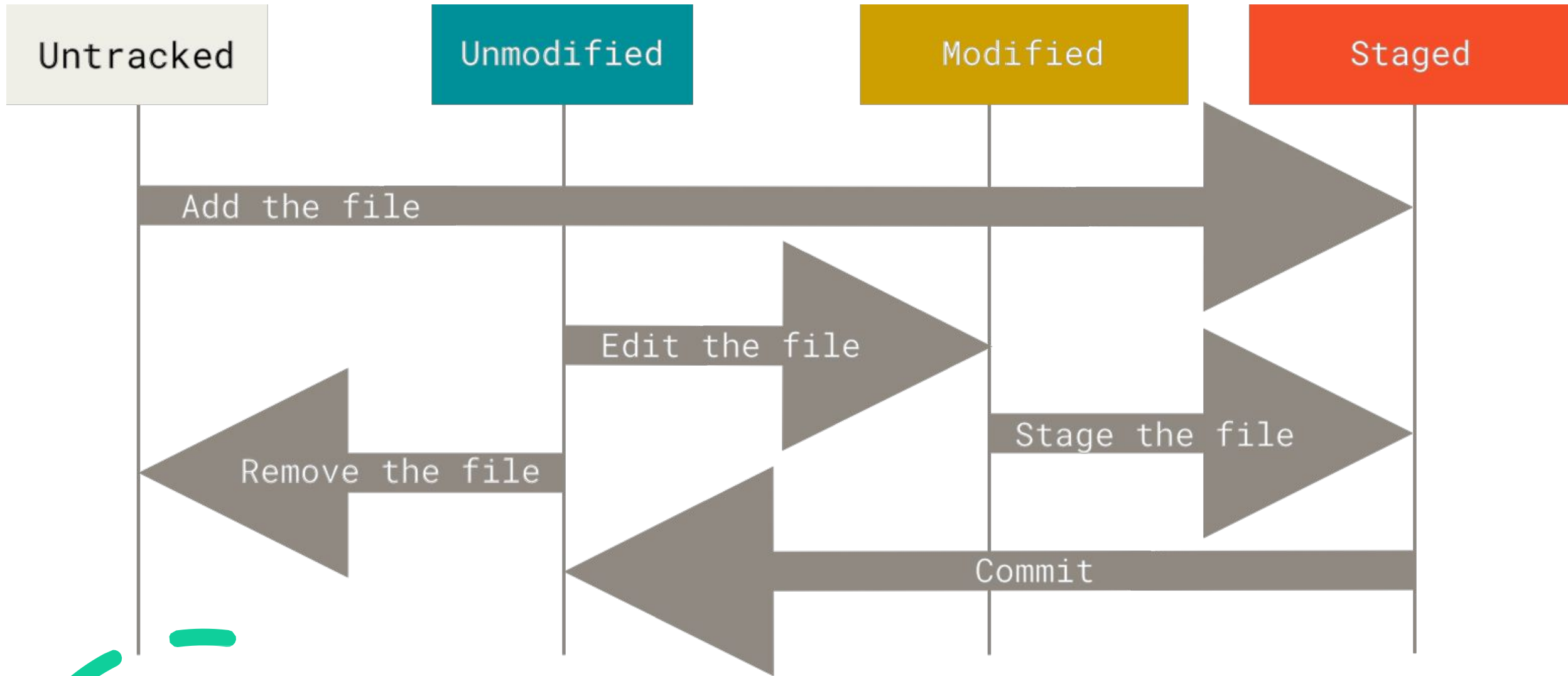
Issue Linking converts issue numbers in commit messages into

Crear un repositorio



Aparece una carpeta oculta con la información del repositorio

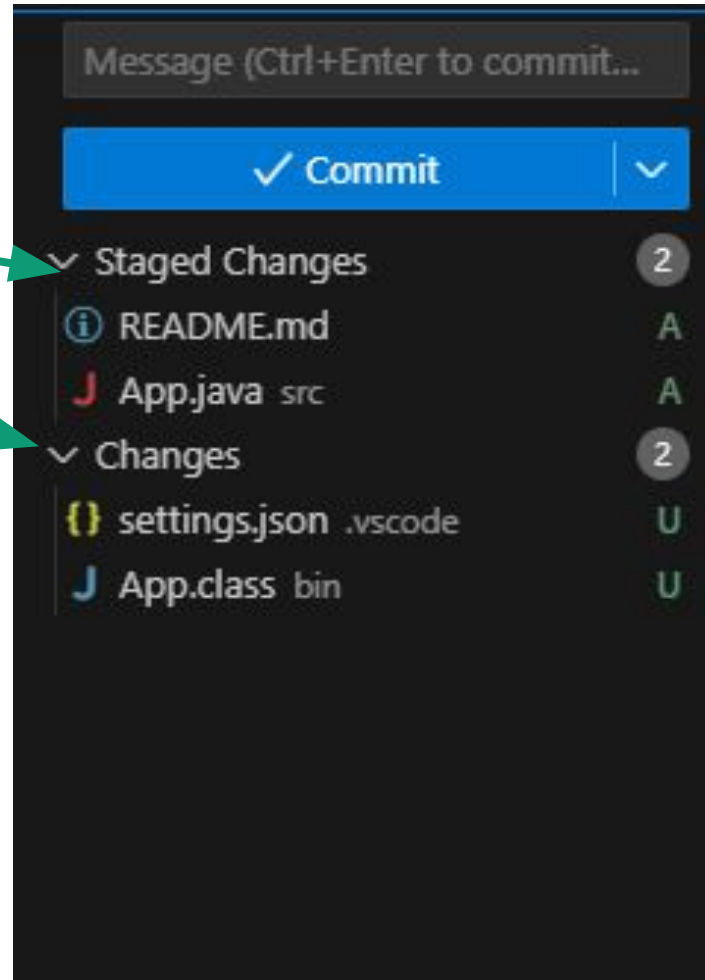
Estados de los archivos en un repositorio



Añadir archivos

Ficheros añadidos

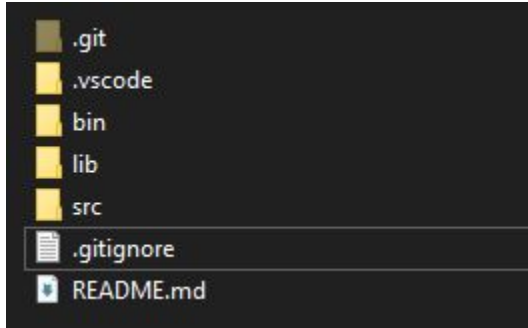
Ficheros sin añadir



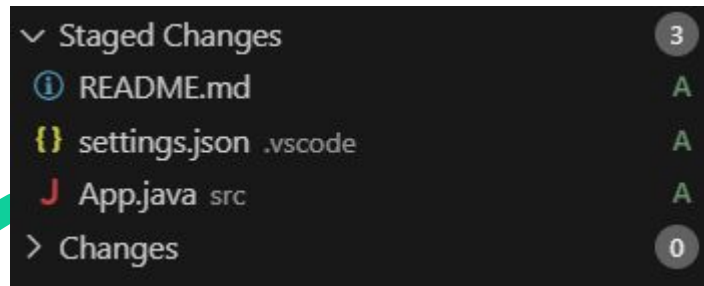
```
#Añadir un archivo al área de preparación
git add <nombre_del_archivo>
#Añadir todos los archivos modificados
git add .
```

.gitignore

Se añade en la raíz del repositorio



Los ficheros (muchas veces se pone un fichero genérico con una extensión) del gitignore no se almacenan en el repositorio. Los .class desaparecen de la lista de cambios



```
# Compiled class file
*.class

# Log file
*.log

# BlueJ files
*.ctxt

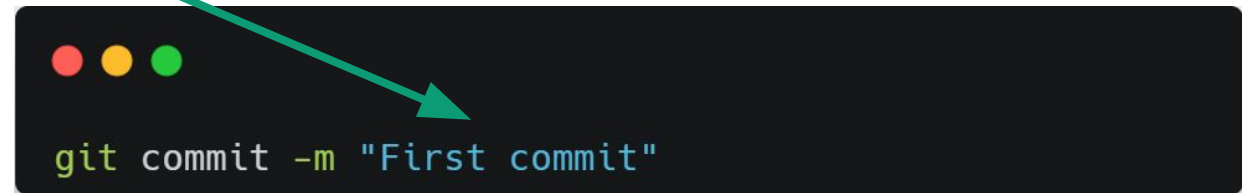
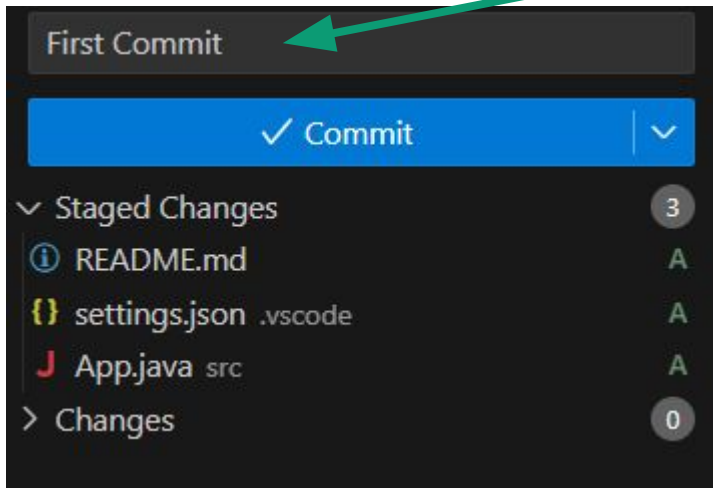
# Mobile Tools for Java (J2ME)
.mtj.tmp/

# Package Files #
*.jar
*.war
*.nar
*.ear
*.zip
*.tar.gz
*.rar

# virtual machine crash logs, see
http://www.java.com/en/download/help/error_hotspot.xml
hs_err_pid*
replay_pid*
```


Crear un commit

Mensaje de commit

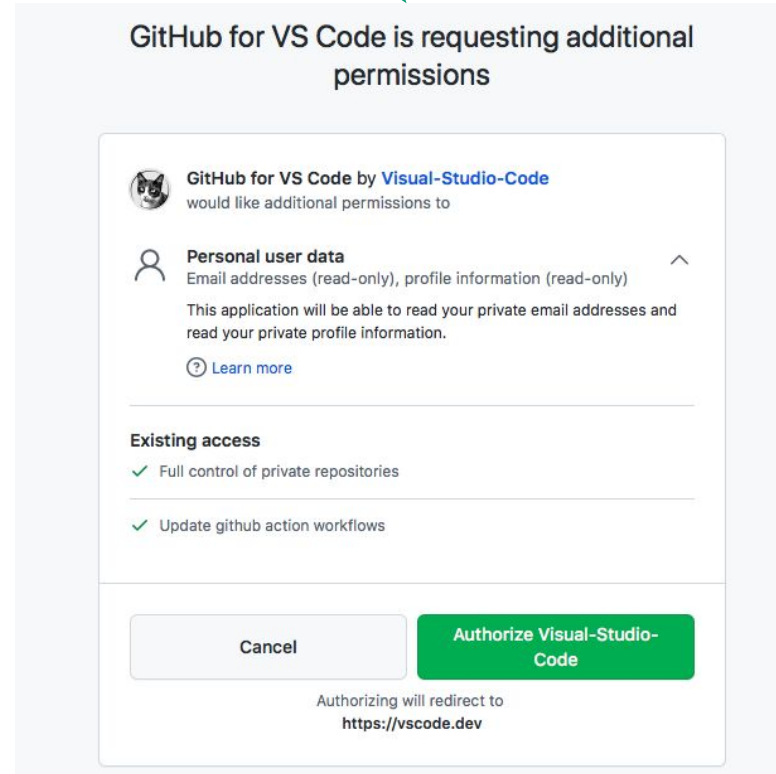
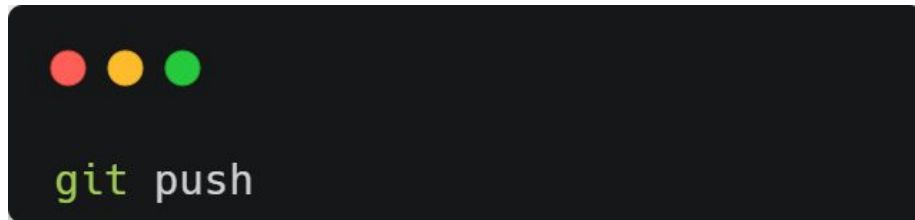
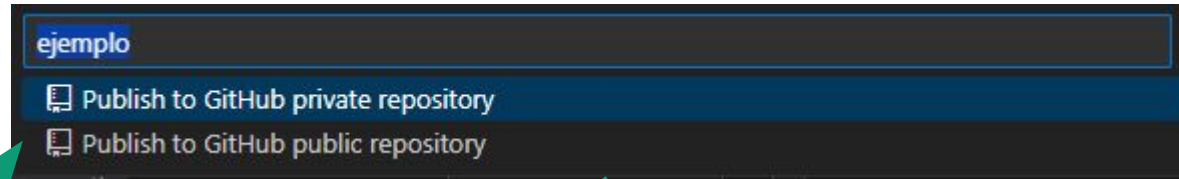
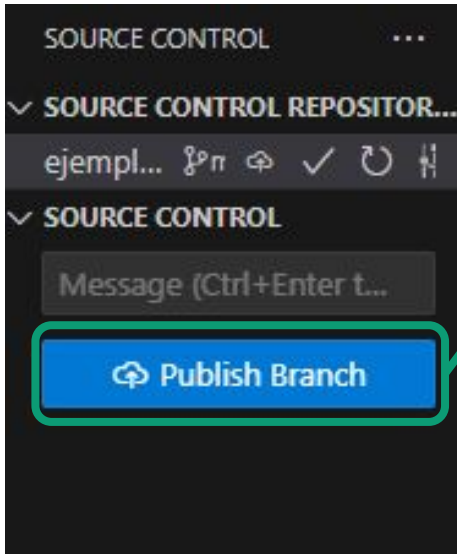


A screenshot of the Git GUI showing the commit history. It includes a "Branches:" dropdown set to "Show All", a "Show Remote Branches" checkbox, and a table of commits.

Graph	Description	Date	Author	Commit
	main First commit	16 Oct 2024...	Serra	0d372ff2
	Add gitignore	16 Oct 2024...	Serra	bf17c208

Un commit es un punto de guardado del repositorio, que se crea con los ficheros preparados en la staged area

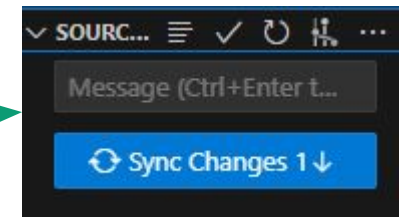
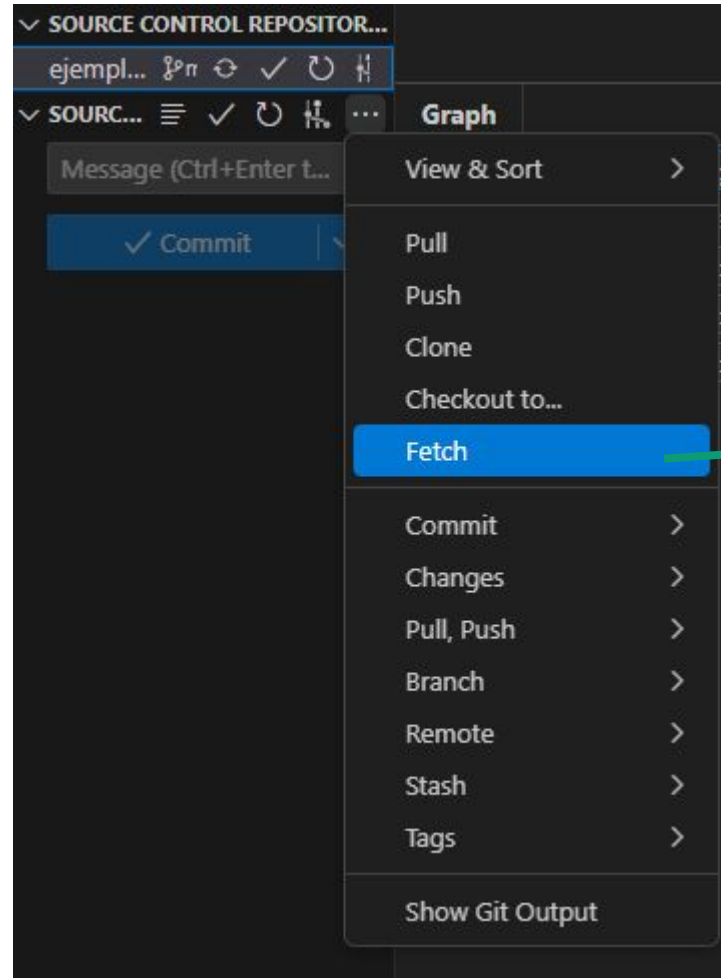
Subir una copia al repositorio remoto



Comprobar y traer cambios del remoto

```
git fetch
```

```
git pull
```



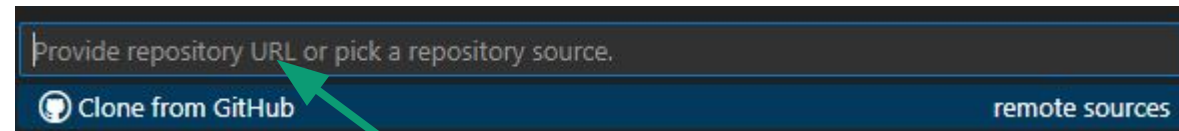
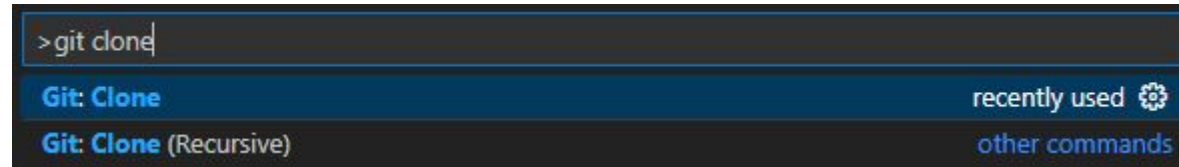
Fetch: Comprobar si hay cambios en el remoto

Pull: Traer cambios del remoto

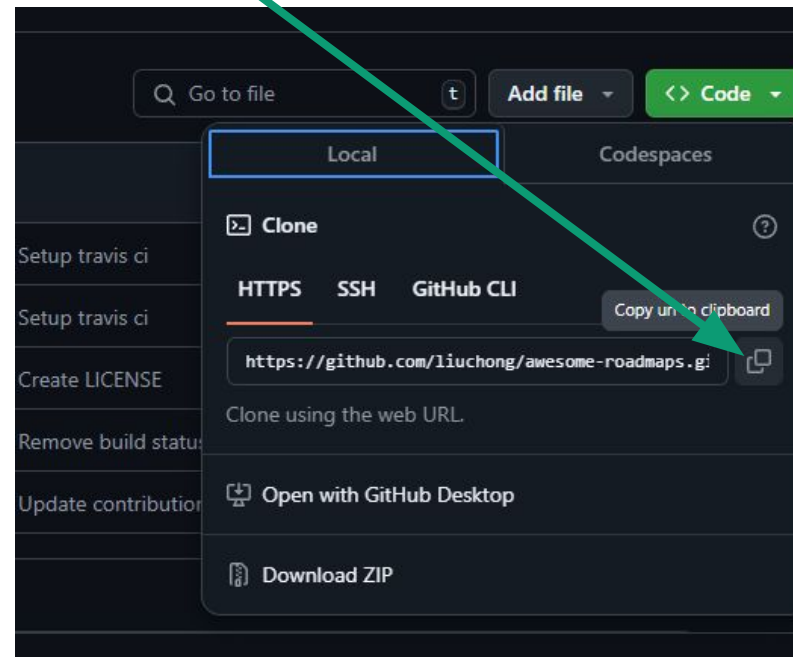
Sync: Subir cambios pendientes + comprobar cambios + traer cambios

Clonar repositorio

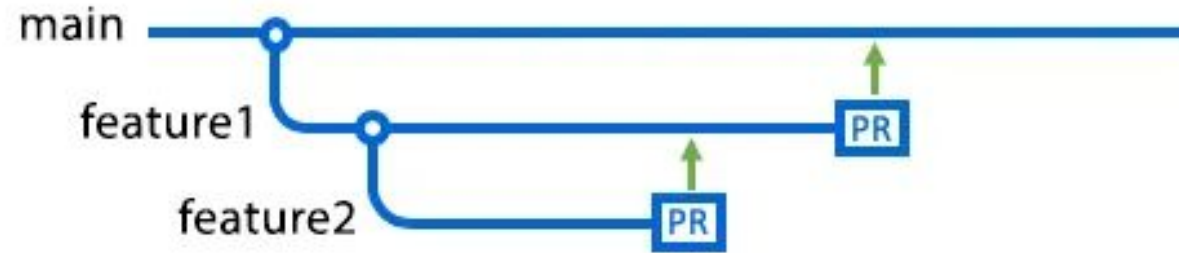
Show and run commands



Se puede clonar un repositorio público mediante el enlace. Si se es dueño o se participa en un repo, aparecen directamente en vscode en la interfaz de clonado.

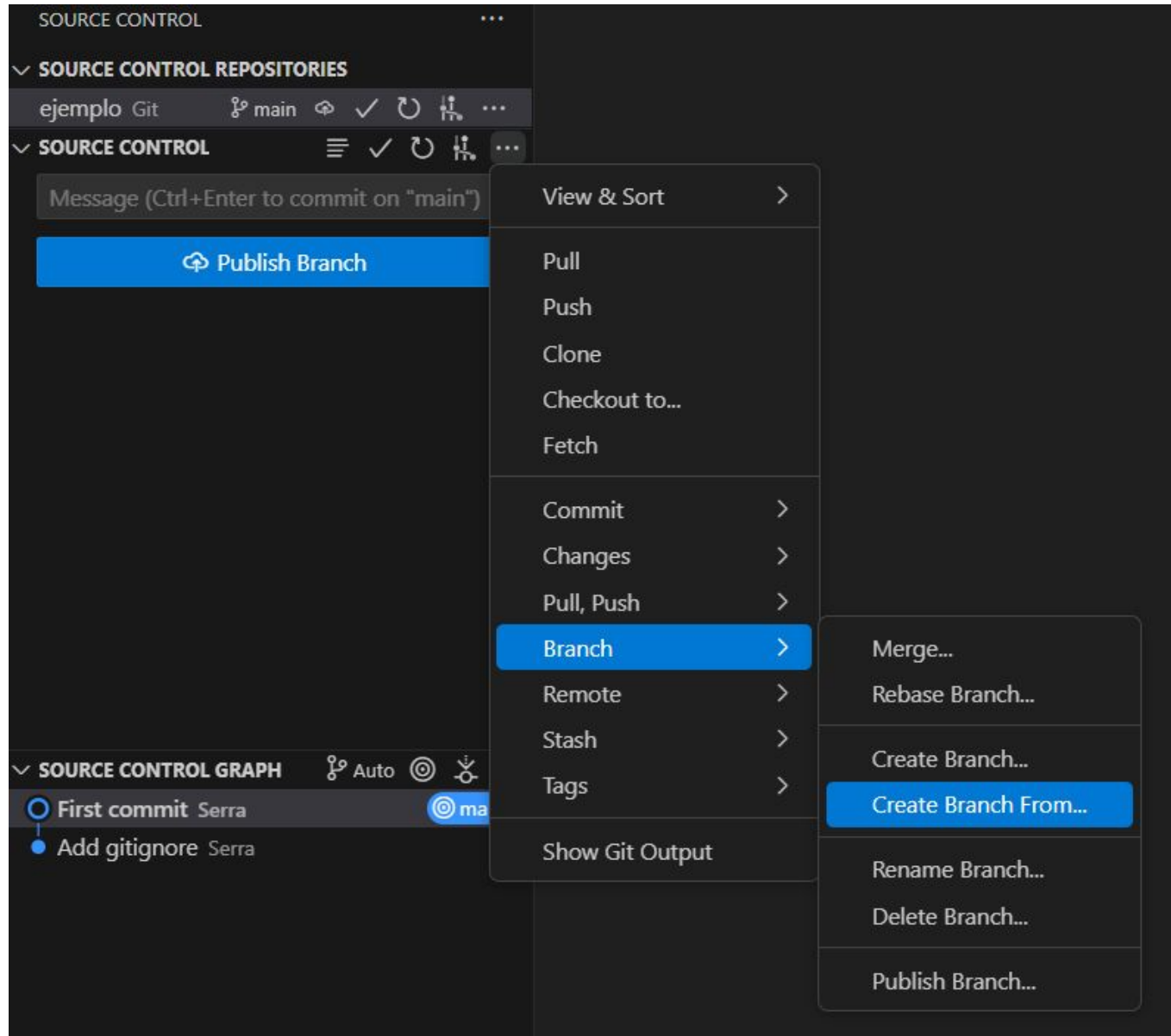


Ramas

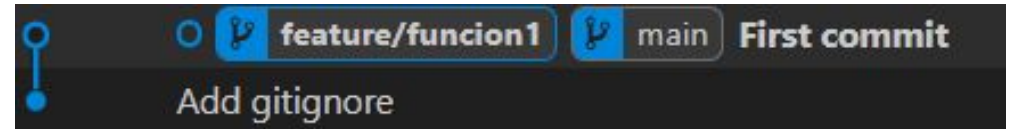


Las ramas permiten desarrollar características, corregir errores, o experimentar sin afectar al resto del proyecto

Crear una rama



```
git branch feature/funcion1
```

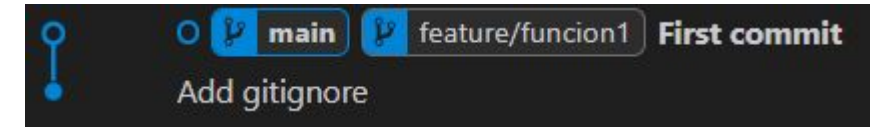
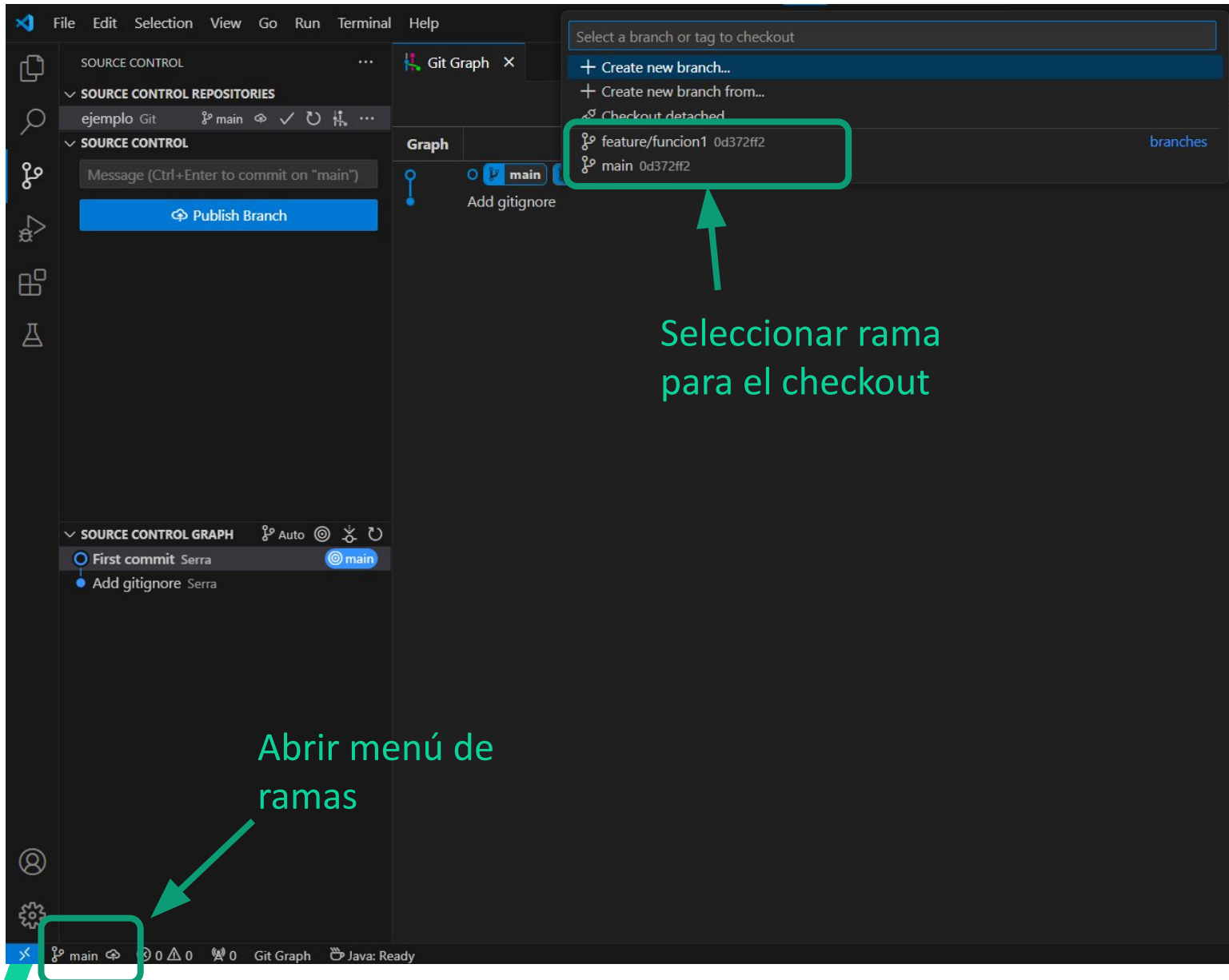


Por defecto existe una rama llamada **main**, donde estará el programa listo para despliegue.

Se recomienda tener una rama llamada **develop**, donde se realiza el desarrollo.

Las distintas funcionalidades del proyecto se realizan en ramas de tipo **feature**.

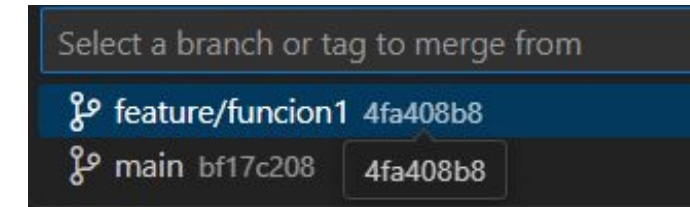
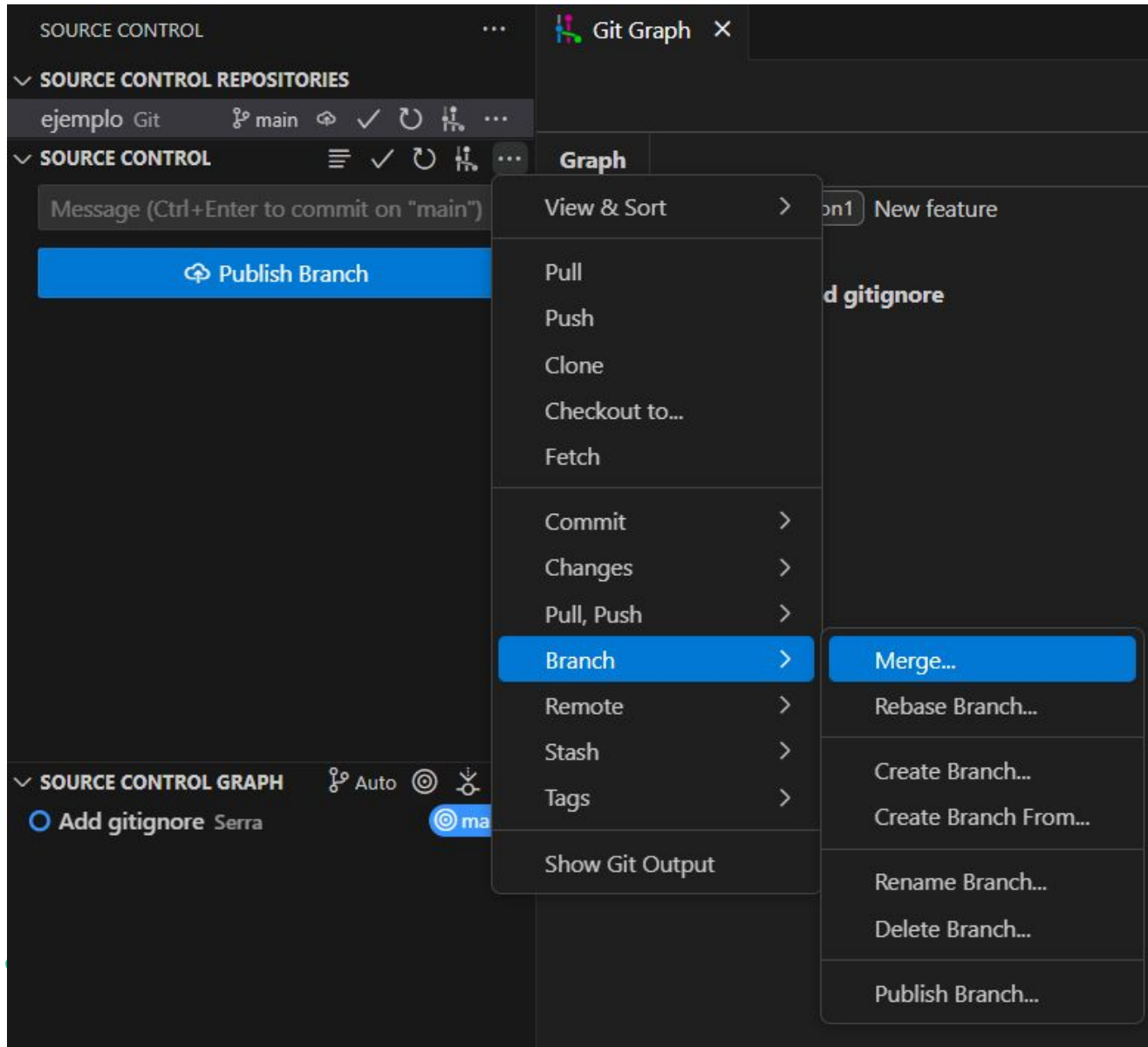
Cambiar de rama



O hacer doble click en la interfaz gráfica de git graph

```
git checkout feature/funcion1
```

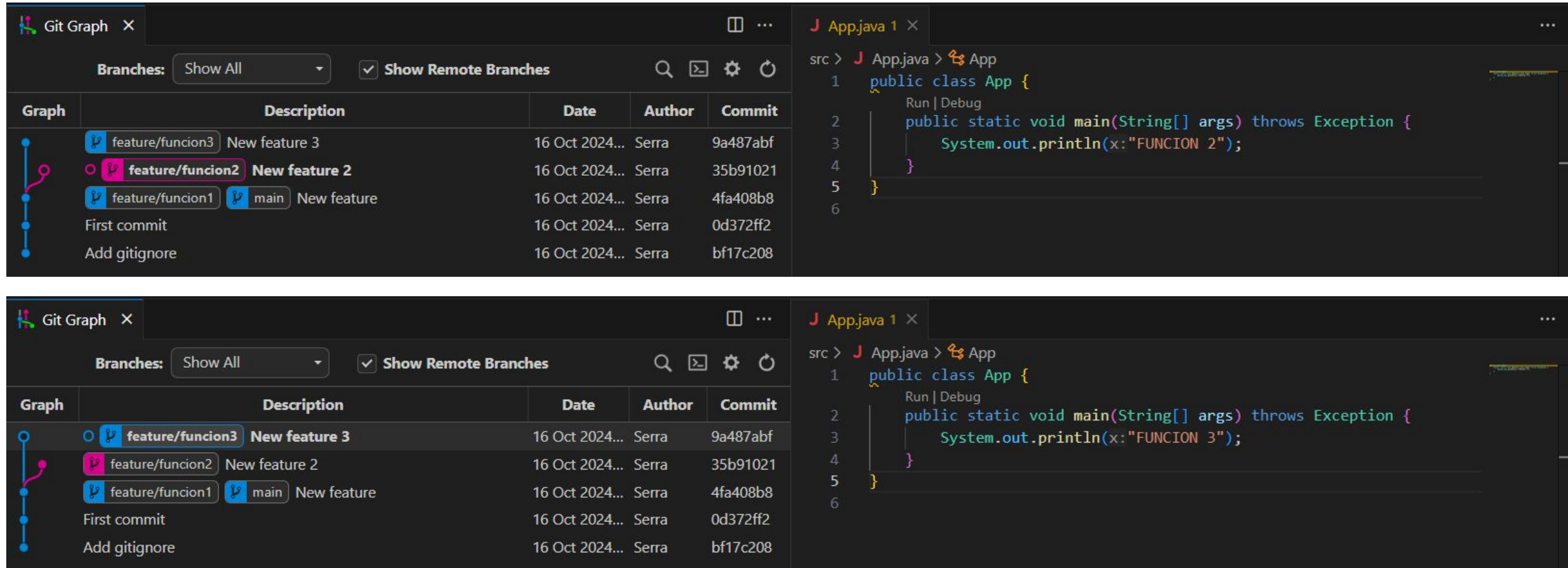
Mezclar ramas



Para llevar los cambios de una rama a la rama main

- Checkout de main
- Seleccionar la opción merge
- Seleccionar la rama desde la que queremos traer los cambios

Conflictos



The screenshot illustrates a Git conflict in Visual Studio Code. The Git Graph view on the left shows a merge of `feature/function2` into `feature/function3`, with a conflict icon. The code editor on the right shows `App.java` with conflicting changes between the two branches.

Graph	Description	Date	Author	Commit
feature/function3	New feature 3	16 Oct 2024...	Serra	9a487abf
feature/function2	New feature 2	16 Oct 2024...	Serra	35b91021
feature/function1	New feature	16 Oct 2024...	Serra	4fa408b8
main	New feature	16 Oct 2024...	Serra	0d372ff2
First commit		16 Oct 2024...	Serra	0d372ff2
Add gitignore		16 Oct 2024...	Serra	bf17c208

```
src > J App.java > App
1 public class App {
2     public static void main(String[] args) throws Exception {
3         System.out.println(x:"FUNCION 2");
4     }
5 }
6
```

```
src > J App.java > App
1 public class App {
2     public static void main(String[] args) throws Exception {
3         System.out.println(x:"FUNCION 3");
4     }
5 }
6
```

Cuando se trabaja simultáneamente en dos ramas distintas sobre los mismos archivos, surge un conflicto a la hora de mezclar ramas

Conflictos

```
1 public class App {
2     Run | Debug
3     public static void main(String[] args) throws Exception {
4         Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
5         <<<<<< HEAD (Current Change)
6             System.out.println(x:"FUNCION 2");
7         =====
8         System.out.println(x:"FUNCION 3");
9         >>>>>> feature/funcion3 (Incoming Change)
10    }
```

El programador tendrá que decidir qué código se mantiene y cual se elimina en caso de conflicto

There are merge conflicts. Resolve them before committing.

Source: Git [Open Git Log](#) [Show Changes](#)

Conflictos

Incoming ϕ 9a487ab · refs/heads/feature/function3

```
1 public class App {
2     public static void main(String[] args) throws Exception {
3         System.out.println("FUNCION 3");
4     }
5 }
6
```

Current ϕ 35b9102 · refs/heads/main, refs/heads/feature/function2

```
1 public class App {
2     public static void main(String[] args) throws Exception {
3         System.out.println("FUNCION 2");
4     }
5 }
6
```

Result src\App.java

Incoming + Current | Remove Incoming | Remove Current

```
1 public class App {
2     public static void main(String[] args) throws Exception {
3         System.out.println(x:"FUNCION 3");
4         System.out.println(x:"FUNCION 2");
5     }
6 }
7
```

0 Conflicts

Complete Merge

Merge branch
'feature/function3'

✓ Commit

✓ Staged Changes

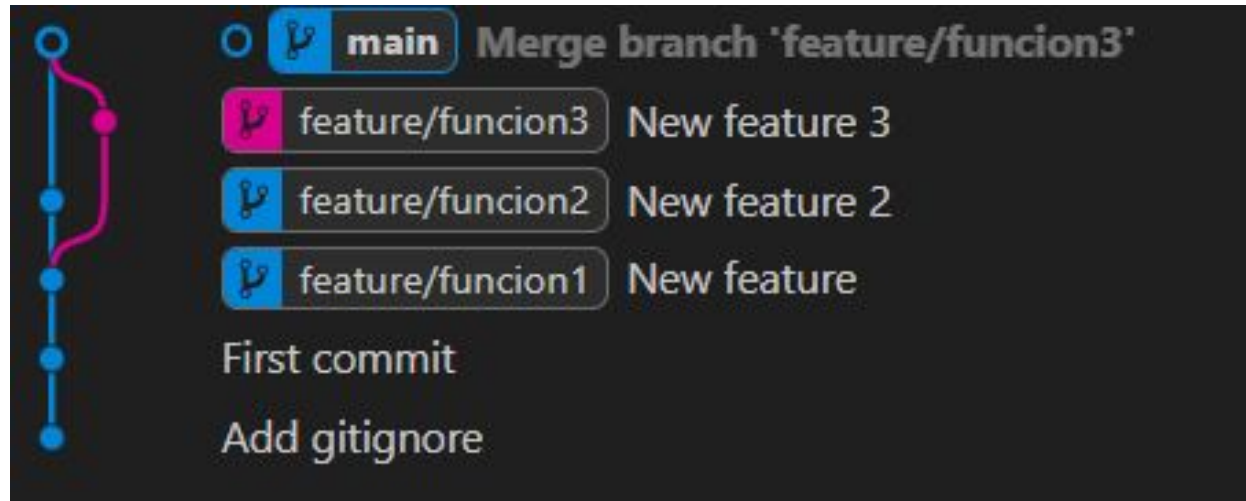
App.java src

Changes

1
M
0

Tras revisar los cambios, habrá que hacer un commit

Conflictos

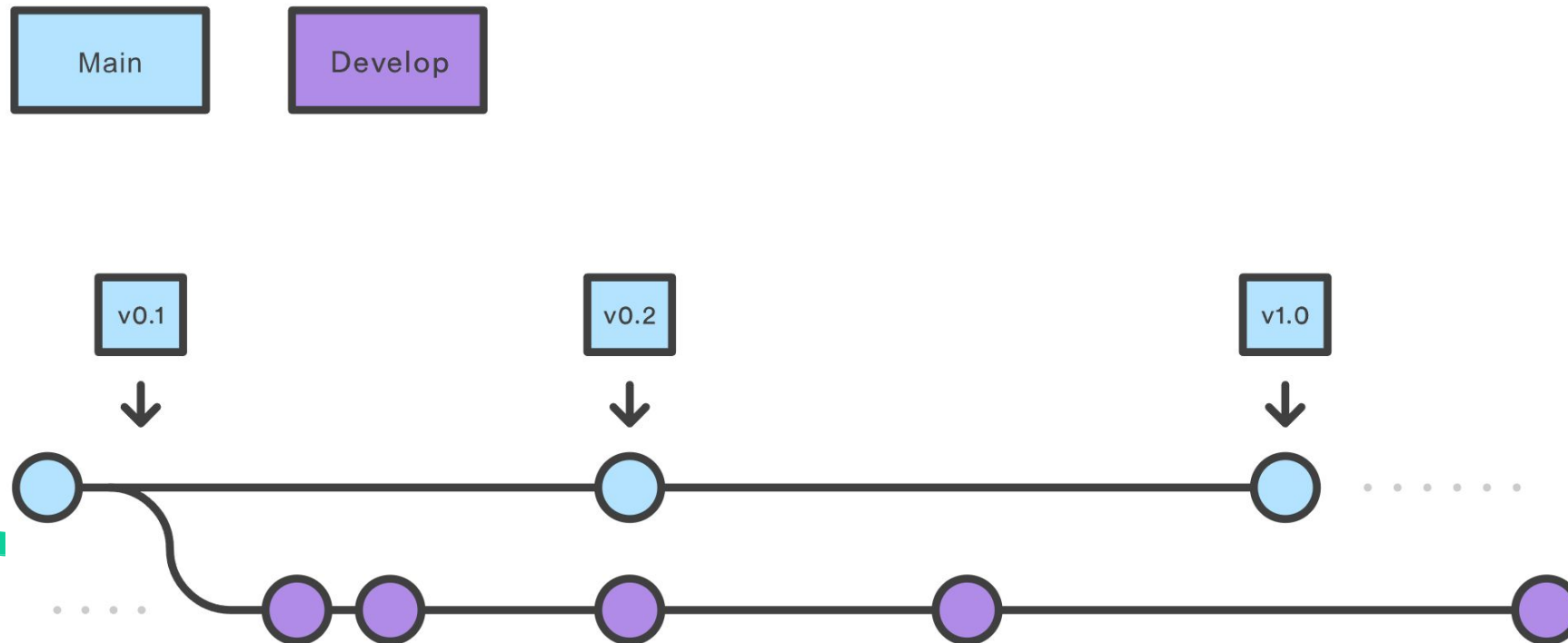


Al final del proceso de merge, tenemos en main tanto los cambios programados en funcion2 como en funcion3

Git Flow

Ramas principales y de desarrollo

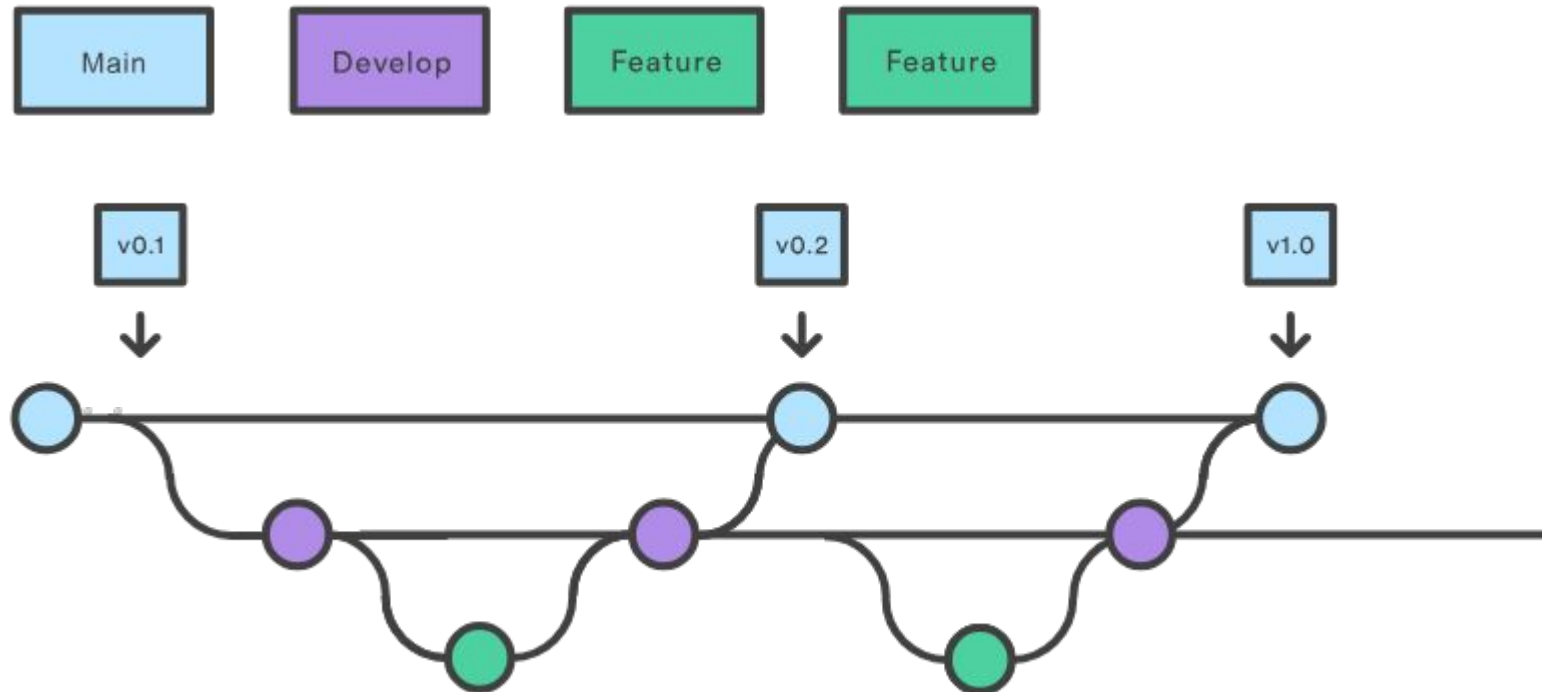
En lugar de una única rama main, este flujo de trabajo utiliza dos ramas para registrar el historial del proyecto. La rama main o principal almacena el historial de publicación oficial y la rama develop o de desarrollo sirve como rama de integración para las funciones. Asimismo, conviene etiquetar todas las confirmaciones de la rama main con un número de versión.



Git Flow

Ramas de función

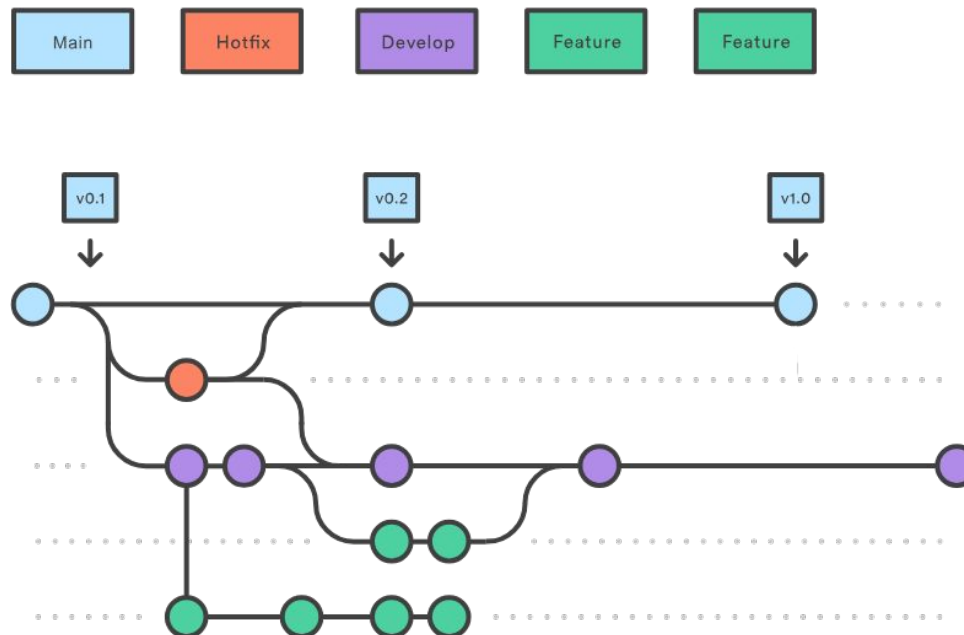
Todas las funciones nuevas deben residir en su propia rama, que se puede enviar al repositorio central para copia de seguridad/colaboración. Sin embargo, en lugar de ramificarse de main, las ramas feature utilizan la rama develop como rama primaria. Cuando una función está terminada, se vuelve a fusionar en develop. Las funciones no deben interactuar nunca directamente con main.



Git Flow

Ramas de corrección

Las ramas de mantenimiento, de corrección o de hotfix sirven para reparar rápidamente las publicaciones de producción. Las ramas hotfix son muy similares a las ramas release y feature, salvo por el hecho de que se basan en la rama main y no en la develop. Esta es la única rama que debería bifurcarse directamente a partir de main. Cuando se haya terminado de aplicar la corrección, debería fusionarse en main y develop, y main debería etiquetarse con un número de versión actualizado.



Git Flow

El flujo general de Git Flow es el siguiente:

- 1 Se crea una rama develop a partir de main.
- 2 Se crean ramas feature a partir de la rama develop.
- 3 Cuando se termina una rama feature, se fusiona en la rama develop.
- 4 Cuando se llega a una versión estable en develop, se integra en main.
- 5 Si se detecta un problema en main, se crea una rama hotfix a partir de main.
- 6 Una vez terminada la rama hotfix, esta se fusiona tanto en develop como en main.